
Eight-dimension non-functional review - lazysite - aggregated overview

0.10.8 EDGE (ec6fe0a), 2026-08-14, Commercial regime -
eight dimensions in signoff order

Friday 14 August 2026

Contents

1	What this is	1
2	Verdicts	2
3	Overall	3
4	The finding behind the findings	4
5	What this period got right	6
6	Required before a stable promotion	7
7	Maintaining compliance	8
8	Recommended, not blocking	9
9	Reports	10
10	Dimension 1 - Correctness and groundedness - lazysite eight-dimension review	11
10.1	Verdict	11
10.2	Method	11
10.3	Findings	12
10.3.1	F1.1 - File::Path::make_path croaks, so the guard after it is unreachable, and a protect call dies with the content still served (REFUSE)	12
10.3.2	F1.2 - The recurring defect class is now being caught by the project itself (PASS, noted)	12
10.3.3	F1.3 - ADR 0001's module-free copies are pinned by behaviour, not by text (PASS)	13
10.3.4	F1.4 - Prior findings verified	13
10.4	Evidence	13
11	Dimension 2 - Code quality - lazysite eight-dimension review	14
11.1	Verdict	14
11.2	Method	14

11.3 Findings	14
11.3.1 F2.1 - The processor is 6,323 lines and carries eleven module-free copies (not, not charged)	14
11.3.2 F2.2 - The tidy gate is changed-code-only, deliberately (PASS - and a note on assessing it)	15
11.3.3 F2.3 - perlcritic severity 3 is clean across the tree (PASS)	15
11.3.4 F2.4 - The lint suite's shape improved, not just its size (PASS)	16
11.3.5 F2.5 - Comments carry cause, not description (PASS, noted)	16
11.4 Evidence	16
12 Dimension 3 - Test coverage - lazysite eight-dimension review	17
12.1 Verdict	17
12.2 Method	17
12.3 Findings	18
12.3.1 F3.1 - The suite passes only where an untracked build artefact happens to exist (WARN, new)	18
12.3.2 F3.2 - Coverage floors are declared, enforced and fail-closed (PASS)	19
12.3.3 F3.3 - The lint suite has become the project's strongest asset (PASS, noted)	20
12.3.4 F3.4 - Tests that were shown to fail first (PASS, noted)	20
12.3.5 F3.5 - The deferred D3/D4 item from July is still open (WARN, carried)	20
12.4 Evidence	20
13 Dimension 4 - Performance - lazysite eight-dimension review	21
13.1 Verdict	21
13.2 Method	21
13.3 Findings	22
13.3.1 F4.1 - The baseline predates two minor lines (WARN)	22
13.3.2 F4.2 - The registries moved onto the request path and are not benchmarked (WARN)	23
13.3.3 F4.3 - The measured baseline documentation is honest about what is stable (PASS, noted)	23
13.3.4 F4.4 - The pending SM294 work measures itself properly (noted, not in the audited tree)	24
13.4 Evidence	24
14 Dimension 5 - Reliability and resilience - lazysite eight-dimension review	25
14.1 Verdict	25
14.2 Method	25

14.3 Findings	25
14.3.1 F5.1 - The restore-rehearsal commitment has lapsed for four stable cycles (WARN)	25
14.3.2 F5.2 - docs/MONITORS.md still does not exist (WARN, carried)	26
14.3.3 F5.3 - A planned-for failure mode was unreachable, so the system failed hard where it was designed to degrade (WARN, cross-referenced to D1 F1.1)	27
14.3.4 F5.4 - What is genuinely solid here (PASS, noted)	27
14.4 Evidence	27
15 Dimension 6 - Security - lazysite eight-dimension review	28
15.1 Verdict	28
15.2 Method	28
15.3 Findings	29
15.3.1 F6.1 - Protected content is served after a crashed protect call (REFUSE)	29
15.3.2 F6.2 - The significant-change register is stale, over its own triggers (REFUSE)	29
15.3.3 F6.3 - The threat model of record predates the architecture (REFUSE)	30
15.3.4 F6.4 - SM283's mitigation is shipped but not deployed where it was disclosed (WARN)	30
15.3.5 F6.5 - What passes (PASS, noted)	31
15.3.6 F6.6 - The SBOM gate cannot be run from the tag it attests (WARN, new)	31
15.4 Evidence	32
16 Dimension 7 - Documentation - lazysite eight-dimension review	33
16.1 Verdict	33
16.2 Method	33
16.3 Findings	34
16.3.1 F7.1 - The feature timeline stops at 0.9.14; the whole 0.10.x line is missing (WARN, regression)	34
16.3.2 F7.2 - docs/POLICY.md cites the 2026-07-01 review as the current one (WARN)	34
16.3.3 F7.3 - ADR 0001's text no longer describes its own arrangement (WARN, carried from D1)	34
16.3.4 F7.4 - The mechanically-defended documentation is the strong part (PASS, noted)	35
16.4 Evidence	35
17 Dimension 8 - Policy compliance - lazysite eight-dimension review	36
17.1 Verdict	36
17.2 Method	36

17.3 Findings	37
17.3.1 F8.1 - Three stable releases have shipped against an unsigned 0.8.0 declaration (REFUSE)	37
17.3.2 F8.2 - The obligations table is itself out of date (WARN, cross-referenced to D7 F7.2)	38
17.3.3 F8.3 - The pentest waiver expiry is approaching and is a policy commitment (WARN)	38
17.3.4 F8.4 - Supply chain and licensing (PASS)	38
17.3.5 F8.5 - Obligations still open, unchanged since July (noted, not new)	38
17.4 Maintaining compliance	39
17.4.1 F8.6 - CRA Article 14 (reporting) is absent from the posture of record (REFUSE)	39
17.4.2 F8.7 - The declared remediation SLAs have no evidence (WARN)	40
17.4.3 F8.8 - Dated obligations are scattered, and one is written relatively (WARN)	40
17.4.4 F8.9 - Annex VII and signed releases get monotonically more expensive (WARN)	41
17.4.5 F8.10 - There is no per-project implementation document (WARN)	41
17.5 Evidence	41

1 What this is

The fourth full eight-dimension non-functional review of lazysite, run against the framework in `/srv/projects/toolchain-development/TOOLCHAIN.md` - the eight dimensions in signoff order, with per-dimension refusal conditions keyed to the declared regime. lazysite declares the **Commercial** regime in `docs/POLICY.md`.

Predecessors: 2026-06-23 (seven-dimension), 2026-07-01, 2026-07-10, and 2026-07-18 (the 0.8.0 stable gate). Each 2026-07-18 finding was verified as fixed or open rather than assumed.

The audited artefact is an EDGE release, not a stable candidate. That distinction matters for how the verdicts should be read: this is a periodic close-out over the 0.8.x, 0.9.x and 0.10.x lines, not a gate on a cut that is about to happen. Where a dimension refuses, it refuses *a stable promotion* - except D1/D6, where the defect is live in a build that is deployed.

The audited tree is main at v0.10.8 (ec6fe0a), assessed in a clean worktree. Two branches awaiting `vcs-review` are **not** in the audited tree and are named where they change a verdict: `claude/sm296-acl-set-crash` and `claude/sm294-front-door-under-the-pool`.

2 Verdicts

#	Dimension	Assessed	Prior	Why
1	Correctness and groundedness	REFUSE	PASS	A croaking 'make_path' makes the guard after it unreachable; a protect call dies leaving content stored-as-protected and still served. Live in 0.10.8. Fix written, on a branch
2	Code quality	PASS	PASS	perlcritic sev-3 clean tree-wide; lint suite 12 -> 41 and improved in kind. Render path up 52% since 0.7.0 - recorded as pressure, not charged
3	Test coverage	WARN	PASS	The suite does not pass on a clean checkout of the tag: 15 test files need 'install.pl', which needs a gitignored build artefact. Coverage floors themselves are enforced and met
4	Performance	WARN	PASS	Baseline captured 2026-07-02, predating two minor lines; 2x tolerance against 3% measured spread; the registries moved onto the request path unbenchmarked
5	Reliability and resilience	WARN	PASS	The restore-rehearsal cadence the declaration itself mandates has lapsed for four stable releases; 'docs/MONITORS.md' still absent
6	Security	REFUSE	REFUSE (cleared)	The D1 defect's consequence is an exposure; the significant-change register is stale over its own triggers; the threat model of record does not mention the private store or the front door
7	Documentation	WARN	WARN (cleared)	FEATURES.md's timeline stops at 0.9.14, eight releases back; POLICY.md cites the 2026-07-01 review as current
8	Policy compliance	REFUSE	WARN	The Declaration of Conformity is unsigned and stamped 0.8.0 with three stable releases shipped since; and CRA Article 14 reporting is absent from the tree entirely, with no named person and no runbook, weeks before it is understood to apply

3 Overall

A stable promotion refuses today, on D1, D6 and D8. One of those is a code defect with a fix already written; the other two are paperwork.

That summary is accurate and, on its own, misleading. It should be read alongside the engineering the period actually contains, because the two are almost opposites.

4 The finding behind the findings

Sort this review's results by whether the thing assessed is **defended by a mechanism** or **maintained by a person**, and the table reorganises itself completely:

Defended by a mechanism	State	Maintained by hand	State
perlcritic + tidy gates	clean	Declaration of Conformity	3 releases stale
Coverage floors ('coverage-floor')	enforced, met	Significant-change register	stale over its own triggers
Strict SBOM gate	right mechanism, un-runnable from the tag	FEATURES.md timeline	8 releases stale
't/lint/36' access model vs code	pinned	STRIDE/ASVS threat model	predates the architecture
't/lint/35'/'37' two-copy parity	pinned	Restore-rehearsal register	4 stable cycles lapsed
Generated capability map	current	Bench baseline	6 weeks, 2 minor lines stale
't/lint/31'/'41' derived lists	pinned	POLICY.md review pointer	2 generations stale

Every column-one item passes. Every column-two item is a finding. Not one mechanised control has rotted; not one hand-maintained record has survived six weeks of a fast release cadence.

This is not a new lesson for this project - it is *the* lesson this project has been teaching itself all period. It found a hand-maintained list silently going stale **four separate times** in the code (t/lint/31's

templates, t/lint/39's scripts, t/lint/41's deb payload, t/lint/34's config list), and each time correctly converted the list into something derived. The same failure is running unchecked through the compliance and documentation records, where nobody has yet made that move.

The recommendation follows directly and is the single most valuable thing in this review: **treat the release records as code, and gate them.** Concretely - a release-gate check that refuses a stable cut when the DoC version is behind the tag; when the significant-change register has no entry covering the SM numbers in the release; when FEATURES.md's newest timeline entry is older than the tag; when the newest rehearsal predates the last stable; when the bench baseline predates the tag. Each is a few lines against data the project already has, and each replaces a person remembering with a build failing. The project has proven it knows how to do this - it has done it four times this month, just not to these files.

5 What this period got right

Recorded deliberately, because a review that lists only defects misrepresents the work.

- **The structural fix was taken rather than patched.** SM248, SM268 H17 and SM283 were one cause: security living in front-end configuration the project ships as a template, cannot test where it is installed, and mostly cannot see. Every previous response had been “put the rule in one more config file”. The 0.10.8 line took the other answer and moved the decisions inside the engine.
- **The defect class was named and hunted.** “A control reports success without doing the work” was identified as a recurring shape and five instances were closed - including one in the project’s own security probe, which had shipped passing by testing nothing.
- **Tests are shown failing first.** Spot-checked repeatedly; `t/lint/33` was verified failing both with the templates absent and with the ACL branch stripped while the observable header stayed.
- **Explanations get falsified.** Running nginx for real disproved a claim the project had made in three places about *why* a protection worked. The protection was real; the reason was wrong; no text match would ever have caught it.
- **Deferred work is filed, not implied.** SM294 shipped items 1, 3 and 4 of its own plan and filed item 2 as SM297 with the argument for not taking it - an auth-spine rewrite where being wrong is an authentication bypass.

6 Required before a stable promotion

1. **Land the SM296 fix** (D1 F1.1 / D6 F6.1). Written and tested on `claude/sm296-acl-set-crash`; awaiting review. Until it ships, the re-apply sweep that the 0.10.8 changelog tells operators to run is the operation that crashes.
2. **Finalise and sign a Declaration of Conformity** for the current stable (D8 F8.1), and add advancing it to the stable-cut procedure.
3. **Write the significant-change register entries** for SM286 (new processing of restricted data) and SM293 step 5 (new external interface) (D6 F6.2). The pentest waiver's validity depends on this register being kept.
4. **Update the threat model of record** to describe the private store and the front door (D6 F6.3).
5. **Run and record one restore rehearsal** against the current stable, then hold or amend the declared cadence (D5 F5.1).

7 Maintaining compliance

The list above restores the records. This list is about not having to write the same review again in six weeks, and one item on it has a nearer date than anything else in this document. Detail in `dimension-8-policy.md` F8.6-F8.10.

- **CRA Article 14 has no owner, no runbook and no mention in the tree** (F8.6). `docs/POL-ICY.md` covers Article 13 thoroughly and Article 14 not at all. The reporting duties are understood to apply from **11 September 2026** - four weeks out - though both that date and the scope judgement want the legal review the DoC already calls for. What is needed is a **named individual**, a **runbook** from “we found a live exposure” to “notified whom, by when”, and a **rehearsal**. SM283 is the worked example: a live exposure across a fleet for weeks, with no mechanism to produce a 24-hour early warning had one been due.
- **The declared remediation SLAs have no evidence** (F8.7). ADR 0007 commits to critical 72h / high 30d / medium 90d / low 180d and nothing records whether any was met. A dated vulnerability register turns behaviour that was in fact good into behaviour that can be shown.
- **Dated obligations are scattered across four documents, and one is relative** (F8.8). The support period should read **2031-07-10** rather than “five years from 0.7.0”. One register, an owner per row, and gate the lead times the same way the records get gated.
- **Annex VII and signed releases get monotonically more expensive** (F8.9). The technical file is mostly *assembly* of artefacts that already exist - start it as an index now and it stays current as a by-product. Signed releases are worse: every release cut unattested is permanently unattestable.
- **There is no per-project implementation document** (F8.10). The framework expects one authored from `IMPLEMENTATION-TEMPLATE.md`; its monitors and migration-plan sections are precisely the two gaps found elsewhere in this review, so writing it would have surfaced them without a review.

8 Recommended, not blocking

6. Make the tag self-sufficient (D3 F3.1, D6 F6.6). `release-manifest.json` is a gitignored build artefact, and **two** things need it: `install.pl` - so fifteen test files fail on a clean checkout - and the strict SBOM gate, which is a CRA control. A released tag currently fails its own tests and cannot run its own compliance check for anyone who clones it. One fix closes both.
7. Re-capture the bench baseline at each stable cut, add a registry-generation op, and add a warn-level tolerance band (D4 F4.1, F4.2). The run on this tree passed while sitting 7.5-27.5% above baseline on every op.
8. Retire the three branch-floor overrides in `dist/config/coverage-floor` - they now measure 64.6-65.6% against a 62% general floor, which is the retirement condition the file states for itself (D3 F3.2).
9. Sweep `FEATURES.md` to the current release and fix `POLICY.md`'s review pointer (D7 F7.1, F7.2).
10. Confirm SM283's proxy template is deployed fleet-wide (D6 F6.4, tasks #204 and #196) - it is absent on `edge.explore.lazysite.io`, the host the disclosure came through.
11. Book the third-party pentest engagement: ADR 0007's waiver expires 2026-12-31 or at first GA marketing, whichever comes first (D8 F8.3).

9 Reports

- `dimension-1-correctness.md`
- `dimension-2-code-quality.md`
- `dimension-3-test-coverage.md`
- `dimension-4-performance.md`
- `dimension-5-reliability.md`
- `dimension-6-security.md`
- `dimension-7-documentation.md`
- `dimension-8-policy.md`

10 Dimension 1 - Correctness and groundedness - lazysite eight-dimension review

- Audited tree: main at v0.10.8 (ec6fe0a), worktree /srv/projects/lazysite-audit
- Date: 2026-08-14
- Regime: Commercial
- Prior verdict: PASS (2026-07-18, 0.8.0 gate)

10.1 Verdict

REFUSE at the audited tree, for one reproducible defect that is live in the released 0.10.8 edge build (F1.1). The remedy exists and is written, on a branch awaiting review; it is not in the audited tree, so the audited tree refuses.

Everything else assessed on this dimension passes. The full suite is green (357 files, 7336 tests), no symbol or API groundedness problem was found in the 0.9.x/0.10.x work, and the programme that dominates this period - SM285 to SM293 - is unusually well grounded, because much of it consists of the engine being made to *check* claims it previously asserted.

10.2 Method

- Ran the full suite on the audited tree (prove -lr t/).
- Verified each 2026-07-18 finding as fixed or open rather than assuming it.
- Read the SM285-SM296 filings against the code they claim to have changed, concentrating on the recurring defect class this project has named for itself: a control that reports success without doing the work.
- Checked the two module-free copies ADR 0001 requires against the modules they mirror, via the lints that pin them (t/lint/35, t/lint/37).

10.3 Findings

10.3.1 F1.1 - File::Path::make_path croaks, so the guard after it is unreachable, and a protect call dies with the content still served (REFUSE)

lib/Lazysite/Private.pm:223 and :261:

```
1 make_path($parent) unless -d $parent;  
2 return ( 0, 'cannot create the private store' ) unless -d $parent;
```

That reads as though failure arrives as a false return. It does not: `make_path` **croaks**. The guard on the second line is therefore unreachable, and the die goes straight out through `action_acl_set`, past the warning and past the audit write.

The state it leaves is the reason this refuses rather than warns. The ACL is saved *before* the move, so after a crashed call the stored rule is complete and `acl-get` returns it; pages answer 302 to an anonymous request; **static files answer 200, byte-identical to the source**; and the call is absent from the audit trail, because the process died before the audit write - so the trail and the stored ACL disagree about whether anyone protected that content.

A protected folder gating its pages and serving its images is SM283 exactly, reached through the mechanism built to make that structurally impossible.

Reported from a 0.10.8 site over both partner surfaces (MCP `set_permissions` returned -32603, control API `acl-set` returned HTTP 500), and reproducible in-tree. `Lazysite::Private`'s own invariant held throughout - the content was in exactly one tree, and the failure direction was "not moved" rather than "in both" - so nothing was lost or duplicated.

Remedy: filed as SM296 and implemented on `claude/sm296-acl-set-crash` (`_mkpath` captures the error and returns; `make_path` is no longer imported into that module, because an unqualified call is how this happened). Not in the audited tree.

10.3.2 F1.2 - The recurring defect class is now being caught by the project itself (PASS, noted)

Worth recording because it is the trend that matters more than any single finding. Five instances of "a control reported success without doing the work" were found and closed in this period - SM278, SM283, SM285's own probe, SM291 and SM296 - and in three of those the finder was the project's own test or check rather than a user.

The clearest case is SM285: `lazysite check --check-acl` shipped with its extension list in a file-scoped my **below** the main body, which exits before reaching it. The list was empty, so the probe made zero fetches, compared `0 == 0`, and reported "the front end respects the ACL" against a port with nothing listening. A security check that passes by testing nothing is precisely the defect the

programme exists to remove. It has a regression test, and `t/lint/39` now refuses state below the dispatch generally.

10.3.3 F1.3 - ADR 0001's module-free copies are pinned by behaviour, not by text (PASS)

The 2026-07-18 review left a WARN on ADR 0001 saying “one recorded copy” while two existed. At the audited tree there are more copies, not fewer - but each is now pinned by a lint that **drives both implementations and compares answers** rather than diffing source: `t/lint/35` (group resolution), `t/lint/37` (engine-dir resolution). That is the stronger form, since two implementations that read alike can still disagree.

The ADR text has not been updated to describe the current arrangement, which is a documentation item rather than a correctness one - carried to D7 as F7.3.

10.3.4 F1.4 - Prior findings verified

Prior finding	State at this tree
F6.10 stored XSS via front-matter <code>lang</code> :	Fixed, regression test present
F6.11 <code>domain-add</code> CRLF	Fixed
Six <code>:utf8</code> fail-open readers	:raw, with the regression test still passing
ADR 0001 “one recorded copy”	Still inaccurate as text; see F1.3 / F7.3

10.4 Evidence

- `prove -lr t/` on `ec6fe0a`: 357 files, 7336 tests, Result: PASS.
- `lib/Lazysite/Private.pm:223, :261` - the croaking call.
- `docs/feature-requests/SM296-protecting-content-crashed-and-left-it-served.md`.
- `t/lint/35`, `t/lint/37`, `t/lint/39` - the pinning and the trap check.

11 Dimension 2 - Code quality - lazysite eight-dimension review

- Audited tree: `main` at `v0.10.8` (`ec6fe0a`)
- Date: 2026-08-14
- Regime: Commercial
- Prior verdict: PASS (2026-07-18)

11.1 Verdict

PASS, with one structural pressure recorded rather than charged as a defect (F2.1). The mechanical gates are clean, the lint suite has more than tripled since the 0.8.0 gate, and the *kind* of lint the project writes has improved - from matching text to driving two implementations and comparing their answers.

11.2 Method

- `perlcritic --profile .perlcriticrc --severity 3` across every CGI, lib/ and tools/.
- Established what the project's tidy gate actually requires before assessing against it - see F2.2, which is a note about the audit rather than the code.
- Measured the growth of the render path across releases, because ADR 0001 concentrates complexity there by design and that pressure is worth tracking.

11.3 Findings

11.3.1 F2.1 - The processor is 6,323 lines and carries eleven module-free copies (noted, not charged)

ADR 0001 keeps Lazysite modules out of the render path, so anything the processor needs it must carry itself. That is a deliberate and defensible trade - it is why the render path is fast and why it has no load-order surprises - but it has a cost that compounds, and the cost is now visible:

Tag	lazysite-processor.pl	Perl files
v0.7.0	4,151	49
v0.8.0	5,178	68
v0.9.10	5,139	68
v0.10.0	5,355	69
v0.10.8	6,323	75

A 52% growth in the single file that is hardest to test in isolation and most expensive to get wrong. This is **not** charged as a defect, for a specific reason: every duplicated decision in that file is now pinned by a lint that drives both copies and compares the answers (t/lint/35 group resolution, t/lint/37 engine-dir resolution, and t/lint/42 on the pending SM294 branch). Duplication that cannot silently diverge is a different thing from duplication.

What should be tracked is the trajectory. The mitigation is per-copy: each new module-free copy needs its own parity lint, and the day one ships without it is the day the ADR's cost becomes a defect. That is worth a standing item rather than a finding.

11.3.2 F2.2 - The tidy gate is changed-code-only, deliberately (PASS - and a note on assessing it)

t/lint/06-tidy.t runs tools/tidy-check.pl, which flags only lines a change touched since the last release tag. The header states the intent plainly: “The existing tree keeps its hand-formatting.”

Recorded here because the audit initially ran perltidy across the whole tracked tree and produced a list of “not tidy” files. That list was **not** a finding - it was measuring a standard the project has explicitly decided against. The check was discarded. Noted so a future assessor does not repeat it and so the deliberate position is on the record.

11.3.3 F2.3 - perlcritic severity 3 is clean across the tree (PASS)

```
1 $ perlcritic --profile .perlcriticrc --severity 3 lazysite-*.pl lib
  tools
2 ... source OK (every file)
3 critic rc=0
```

Clean across all seven CGI surfaces, lib/, and tools/, on a tree that has grown by roughly 1,000 lines in the render path alone since the 0.8.0 gate.

11.3.4 F2.4 - The lint suite's shape improved, not just its size (PASS)

41 lint files, up from 12 at the 0.8.0 gate. The important change is in kind:

- **Behavioural parity** rather than text comparison: `t/lint/35` and `t/lint/37` execute both implementations and compare results, because two implementations that read alike can still disagree.
- **Real tools rather than assumptions**: `t/lint/34` starts `nginx` and parses every shipped config; `t/lint/07` runs `shellcheck`.
- **Derived lists rather than maintained ones**: `t/lint/31` and `t/lint/41` assert that a hand-kept list matches the filesystem, so a template or a surface nobody listed fails rather than passing silently.

The last of these is the project's most valuable recurring lesson, found four separate times and converted into a mechanism each time.

11.3.5 F2.5 - Comments carry cause, not description (PASS, noted)

A quality worth recording because it is unusual and it is load-bearing for a codebase this size. The comments in the changed code explain *why the code is this shape and what went wrong when it was another shape* - `make_path` is not imported into `Private.pm` because "an unqualified call is how this happened"; the extension probe is a sub rather than a file-scoped my because of a named prior incident. That is the form of comment that survives contact with a later maintainer.

11.4 Evidence

- `perlcritic --profile .perlcriticrc --severity 3 -rc0`.
- `t/lint/06-tidy.t`, `tools/tidy-check.pl` - the changed-code-only scope.
- `git show <tag>:lazysite-processor.pl | wc -l` across five tags.

12 Dimension 3 - Test coverage - lazysite eight-dimension review

- Audited tree: `main` at `v0.10.8` (`ec6fe0a`)
- Date: 2026-08-14
- Regime: Commercial
- Prior verdict: PASS (2026-07-18)

12.1 Verdict

WARN. The suite is large, disciplined and genuinely load-bearing - it has caught several of this period's defects before a user did, which is the only real measure of a test suite. Coverage floors are enforced and met.

One finding stops this being a clean pass, and it is a reproducibility problem rather than a coverage one: **the suite does not pass on a clean checkout of the released tag** (F3.1). Fifteen test files invoke `install.pl`, which requires `release-manifest.json` - a gitignored build artefact. On a developer's working copy that file is lying around from the last build and everything passes; in a fresh clone or worktree of `v0.10.8`, it is absent and those tests fail. The release gate has never noticed because the gate runs where the artefact exists.

12.2 Method

- Ran the full suite on a **clean worktree** of the tag rather than in the working copy. This was not a deliberate experiment - it is how the audit was set up, and it is the reason the finding surfaced at all.
- Reproduced the failure to root cause rather than accepting the symptom.
- Ran `tools/coverage.sh --check` against the declared floors.
- Read the lint suite's growth and its shape.

12.3 Findings

12.3.1 F3.1 - The suite passes only where an untracked build artefact happens to exist (WARN, new)

Full suite on a clean worktree of v0.10.8:

```
1 Files=356, Tests=7325 ... Result: FAIL
2 t/tools/38-migrate-engine-tree.t (Wstat: 512 (exited 2) Tests: 7
   Failed: 2)
```

The same file in the primary working copy at the same commit:

```
1 t/tools/38-migrate-engine-tree.t .. ok
2 Result: PASS
```

Root cause, reproduced directly:

```
1 $ perl /srv/projects/lazysite-audit/install.pl --docroot ... --cgibin
   ...
2 release-manifest.json not found at
   /srv/projects/lazysite-audit/release-manifest.json
3
4 $ git ls-files release-manifest.json | wc -l
5 0
6 $ git check-ignore -v release-manifest.json
7 .gitignore:42:release-manifest.json      release-manifest.json
```

`release-manifest.json` is generated at build time and gitignored. `install.pl` requires it. **Fifteen test files invoke `install.pl`**, so the exposure is wider than the two subtests that happened to fail here - those two failed because they check the *result* of the install; others may be tolerating a failed install more quietly.

Why this matters beyond tidiness:

- The suite's result depends on **untracked state in the developer's working directory**. That is the same class as the fixture-agrees-with-the-reader problem this project has already named for itself - the test and the thing it depends on are both under the author's hand, so agreement proves nothing.
- Anyone who clones at v0.10.8 and runs the suite - a reviewer, a CI runner, a contributor, an auditor - gets failures on a released tag.
- It also means the release gate cannot distinguish "the installer works" from "the installer works given a manifest from a previous build".

Remedy options, in preference order: have `install.pl` generate the manifest when absent (or the test fixture generate one); or have the affected tests build it via `tools/build-manifest.pl`, which already exists and is tested by `t/tools/01`; or, weakest, skip with an explicit reason so the gap is visible rather than silent. A `skip` is the least good because it would hide the installer from the suite entirely on clean checkouts.

12.3.2 F3.2 - Coverage floors are declared, enforced and fail-closed (PASS)

`tools/coverage.sh --check` on the audited tree, full suite under `Devel::Cover` with subprocess CGIs instrumented - `rc=0`:

Surface	Statements	Branches	Floors
<code>lazysite-oauth.pl</code>	99.4%	94.8%	75 / 62
<code>lazysite-dav.pl</code>	93.5%	74.5%	75 / 62
<code>tools/lazysite-users.pl</code>	91.9%	74.8%	75 / 62
<code>lazysite-mcp.pl</code>	90.9%	65.4%	75 / 60
<code>tools/lazysite-bundle-apply.pl</code>	89.8%	65.0%	75 / 62
<code>lazysite-processor.pl</code>	88.4%	73.1%	75 / 62
<code>lazysite-auth.pl</code>	82.5%	64.6%	75 / 60
<code>lazysite-manager-api.pl</code>	80.8%	65.6%	75 / 60

Every measured production CGI clears both floors. The floors are also **fail-closed** - an unmeasured file fails rather than passing silently, which is the property that matters more than any individual number.

The three branch-floor overrides are now unnecessary and should be retired. `dist/config/coverage-floor` holds `lazysite-manager-api.pl`, `lazysite-auth.pl` and `lazysite-mcp.pl` at 60 rather than the general 62, because at the v0.6.10 measurement they cleared 62 by less than the documented run-to-run variance. They now measure **65.6, 64.6 and 65.4** - between 2.6 and 3.4 points clear of 62. The file states its own retirement condition ("remove these overrides when ... targeted branch tests lift them clear of 62+variance") and that condition is met. Ratcheting them away is the file's own instruction and costs nothing.

`dist/config/coverage-floor` declares 75% statements and 62% branches per cleanly-measured production CGI, with three documented per-file branch overrides at 60 (`lazysite-manager-api.pl`, `lazysite-auth.pl`, `lazysite-mcp.pl`), each justified in the file by measurement variance rather than by convenience, and each annotated "never lower the others".

That file is a model of how a threshold should be recorded: it states the baseline measurements, the date and review that set them, the reason for every override, the condition under which an override may be relaxed further, and the instruction to ratchet upward only. The statement floor **is** the Commercial floor, met by enforcement rather than by luck.

12.3.3 F3.3 - The lint suite has become the project's strongest asset (PASS, noted)

41 lint files at the audited tree, up from 12 at the 0.8.0 gate. More importantly, their *shape* has changed. The best of them do not match text; they drive two implementations and compare answers (`t/lint/35`, `t/lint/37`), start a real nginx and parse shipped configs (`t/lint/34`), or assert that a hand-kept list matches the filesystem (`t/lint/31`, `t/lint/41`).

The project has now found the same defect - **a hand-maintained list that silently went stale** - four separate times, and each time converted it into a derived list. That is the correct response and it is visibly working.

12.3.4 F3.4 - Tests that were shown to fail first (PASS, noted)

Spot-checked several of this period's security and behaviour tests for evidence that they were demonstrated failing against unfixed code before being accepted - `t/lint/33` was verified failing both with the templates absent and with the ACL branch stripped while the header stayed; `t/integration/42` was verified by breaking the template five ways. That practice is the difference between a test and a comment, and it is being followed.

12.3.5 F3.5 - The deferred D3/D4 item from July is still open (WARN, carried)

"`lang_status`'s content walk unbenchmarked; retain a release-suite log before the cut" was deferred at the 0.8.0 gate and has not been done.

12.4 Evidence

- `prove -lr t/` on a clean worktree of `ec6fe0a`: 356 files, 7325 tests, FAIL.
- `install.pl` error output, reproduced above.
- `git check-ignore -v release-manifest.json`.
- `dist/config/coverage-floor`.

13 Dimension 4 - Performance - lazysite eight-dimension review

- Audited tree: main at v0.10.8 (ec6fe0a)
- Date: 2026-08-14
- Regime: Commercial
- Prior verdict: PASS (2026-07-18)

13.1 Verdict

WARN. The benchmark gate exists, is committed, is host-relative with recorded provenance, and runs in the release gate - which is more than most projects of this size have. But it has stopped tracking the system it measures: the baseline is six weeks and two minor lines old (F4.1), and the one structural change in this period that moved work onto the request path - the registries becoming generated-on-request - has no benchmark op at all (F4.2).

Combined with a 2x tolerance, the gate would currently pass a system that had become 90% slower on every measured path while saying nothing about the paths it does not measure.

13.2 Method

- Read `dist/config/bench-baseline.json` and `tools/bench.pl`'s gate logic.
- Compared the four benchmarked ops against the request paths the 0.10.x work created or changed.
- Ran `tools/bench.pl --check` on the audited tree.

13.3 Findings

13.3.1 F4.1 - The baseline predates two minor lines (WARN)

```
1 "captured_at" : "2026-07-02T08:41:10Z",
2 "host" : "ai-dev",
3 "tolerance" : 2
```

Captured 2026-07-02, before the 0.9.x line and the whole of 0.10.x. The design is deliberately host-relative and expects re-capture on the CI/deploy host, so a stale date is not automatically wrong - but a baseline that predates the work it is gating cannot detect a regression introduced by that work. It can only detect a regression introduced *after the next re-capture*.

The 2x tolerance compounds this. `tools/bench.pl`'s own comment records that “measured spread on a quiet host is ~3 per cent”, which means the gate is set roughly 60 standard deviations wide. That is a sensible guard against host variance and a poor guard against gradual drift: six 12% regressions in a row pass every time, and the seventh is compared against a baseline nobody has re-captured.

This is not hypothetical - the run on the audited tree shows the drift already accumulating. `tools/bench.pl --check` passes (`rc=0`, “all ops within tolerance”), and every single op is slower than the baseline it is checked against:

Op	Baseline (2026-07-02)	Audited tree	Drift
<code>verify_token_ms</code>	32.7 ms	41.7 ms	+27.5%
<code>verify_password_ms</code>	120.8 ms	132.9 ms	+10.0%
<code>render_cache_hit_ms</code>	60.0 ms	65.1 ms	+8.5%
<code>render_miss_ms</code>	83.6 ms	89.9 ms	+7.5%

Against a documented ~3% measured spread, a 27.5% move on the partner hot path is signal, not noise. It may be entirely benign - same host but six weeks of system updates, a different kernel, or genuine added work on the token path - and that is exactly the point: **the gate reports “within tolerance” and therefore nobody has to find out which.** A green gate that is silently absorbing a quarter of a second-order regression is doing less work than its output suggests.

Recommendation, concretely: re-capture at each stable cut, and add a warn-level band (say 1.25x) that reports drift without failing the build, so the 2x line stays as the flake-proof hard stop it was designed to be.

Recommendation: re-capture at each stable cut (it is one command), and consider a second, tighter tolerance that warns rather than fails - so drift is visible without making the gate flaky.

13.3.2 F4.2 - The registries moved onto the request path and are not benchmarked (WARN)

SM293 step 3 stopped writing `sitemap.xml`, `llms.txt`, `robots.txt`, `feed.rss` and `feed.atom` to disk. They are now generated on request and cached. This was a deliberate and well-argued decision - it removes a class of staleness bug and keeps a high-demand file off disk - and the user explicitly accepted mild staleness in exchange.

But it moves a **full content walk** onto a request path that previously served a static file, and none of the four benchmarked ops covers it:

Op	Path
<code>render_cache_hit_ms</code>	a cached page
<code>render_miss_ms</code>	a full page render
<code>verify_token_ms</code>	token credential verification
<code>verify_password_ms</code>	password verification

The cost is bounded by the cache, so the steady state is fine. The uncached case is a site-wide scan triggered by an anonymous, unauthenticated request for a well-known URL - which is also the request a crawler makes, and the one an attacker would repeat after a cache invalidation. Nothing here says it is slow; the point is that nobody has measured it, on the release where it changed.

The July review left “`lang_status`’s content walk unbenchmarked” as a deferred D3/D4 item. That is still open, and this is the same shape: content walks are the project’s characteristic scaling risk and are the thing the bench does not cover.

13.3.3 F4.3 - The measured baseline documentation is honest about what is stable (PASS, noted)

`docs/architecture/performance.md` states its absolute numbers are environment-dependent and names the *relative* figures as the ones to compare across environments (cache-hit vs miss overhead, scan scaling, the token-vs- password gap). That is the right way to publish a benchmark and it is followed consistently.

The module-loading analysis is likewise candid: ~50 ms of the ~58 ms the processor takes to print anything is module loading, and the cache-hit path does 4-8 ms of real work on top of that floor. Naming the floor is what makes the FastCGI work’s value legible.

13.3.4 F4.4 - The pending SM294 work measures itself properly (noted, not in the audited tree)

Recorded because it is the pattern the recommendations ask for. The pooled front door on `claude/sm294-front-door-under-the-pool` was measured like-for-like - the same URL and the same authentication state in both configurations - giving 71.9 ms to 0.53 ms on an anonymous page and 107.3 ms to 96.9 ms on a signed-in one, with the reproduction script committed alongside.

The methodological note matters more than the numbers: the first version of that comparison measured a cached anonymous hit against a signed-in miss and appeared to show a 34% regression that did not exist. A benchmark comparing two different requests is worse than no benchmark, because it produces a number.

13.4 Evidence

- `dist/config/bench-baseline.json` - capture date, host, tolerance.
- `tools/bench.pl:6-8, :114-124` - the gate and its tolerance logic.
- `docs/architecture/performance.md:33-75`.
- `lib/Lazysite/FrontDoor.pm:45` - the retired registry routes.

14 Dimension 5 - Reliability and resilience - lazysite eight-dimension review

- Audited tree: main at v0.10.8 (ec6fe0a)
- Date: 2026-08-14
- Regime: Commercial
- Prior verdict: PASS (2026-07-18); REFUSE at 2026-07-10, cleared by docs/RELIABILITY.md

14.1 Verdict

WARN. The declaration that cleared this dimension in July is intact and still accurate, and the mechanisms behind it work. What has lapsed is the evidence the declaration itself demands: the restore-rehearsal cadence stopped four stable releases ago (F5.1), and the monitor document that the SLOs assume somebody is keeping has never been written (F5.2). Neither is a Commercial refusal condition, but a declared commitment that quietly stopped being met is exactly the failure this dimension exists to notice - and it is the second review in a row to note the monitor gap.

One in-period defect belongs here as well as under D1: a failure mode the design had explicitly planned for was **unreachable in practice** (F5.3).

14.2 Method

- Read docs/RELIABILITY.md against what has actually shipped since it was written, rather than against itself.
- Checked the rehearsal register's cadence claim against the release history in CHANGELOG.md.
- Checked the deferred items the 2026-07-18 review left on this dimension.

14.3 Findings

14.3.1 F5.1 - The restore-rehearsal commitment has lapsed for four stable cycles (WARN)

docs/RELIABILITY.md states the rule itself:

Rehearsals are repeated **at least once per stable release cycle**.

The register's last entry is **2026-07-12** (the 0.7.13 stable cycle). Since then the project has cut four stable releases:

Stable release	Date	Rehearsal recorded
0.8.0	2026-07-18	none
0.9.4	2026-07-19	none
0.9.10	2026-07-21	none
0.10.0	2026-07-27	none

This is not a claim that restore is broken - the mechanism is covered by the failure-mode suite, and the last six rehearsals all completed in under a second of mechanical restore. It is that the RTO of 4 hours and the RPO values are declared to be *backed by timed rehearsals*, and for the four most recent stable releases they are backed by the declaration alone. The 2026-07-18 review already flagged “a 0.7.28 restore rehearsal not yet recorded” as deferred; it was deferred rather than done, and three more cycles have passed.

Remedy: run one rehearsal against the current stable tarball and record it, then either hold the per-cycle cadence or amend the declared cadence to one the project will actually meet. A commitment that is quietly not met is worse than a looser commitment that is.

14.3.2 F5.2 - docs/MONITORS.md still does not exist (WARN, carried)

The SLO section assumes an operator-side monitor “that measures availability”, and the 2026-07-18 review recorded the absent monitor document and capacity test as deferred. Neither has appeared. The file is referenced as an intended artefact and is absent from the tree:

```
1 $ ls docs/MONITORS.md
2 ls: cannot access 'docs/MONITORS.md': No such file or directory
```

Because reliability ownership is explicitly **per implementation** (the 2026-07-04 decision, restated at docs/RELIABILITY.md:16–25), this is not a refusal: the project ships the mechanism and each operator declares their own targets. But an operator taking the reference posture as their posture of record has no shipped guidance on what to monitor to know whether they are meeting it, which makes the reference posture undeliverable as written for exactly the operators who most need it.

14.3.3 F5.3 - A planned-for failure mode was unreachable, so the system failed hard where it was designed to degrade (WARN, cross-referenced to D1 F1.1)

The private-store design anticipated an unmovable store and specified the behaviour precisely:

A failed move does not refuse the rule. The ACL is stored and the engine honours it, so the site is no worse off than before the store existed - but the response says so, because both outcomes look identical to the operator otherwise.

The warning text existed and was correct. It could not be reached, because the guard preceding it sat after a croaking call (D1 F1.1). So the designed graceful degradation became a 500 and an absent audit line.

The resilience lesson is narrower than the bug: **a declared failure mode with no test that exercises it is a comment**. The remedy branch adds one (`t/unit/manager/73`) which blocks the store deterministically with a file where its directory must be - no `chmod`, so it behaves identically for an unprivileged user and for root - and asserts the warning, the stored rule and the untouched content together, because the fix is only correct if all three hold at once.

14.3.4 F5.4 - What is genuinely solid here (PASS, noted)

- The RPO-zero-for-code claim remains true and is structural rather than aspirational: every release is an immutable git tag (ADR 0002).
- `lazysite` check gained a store-usability report on the remedy branch, which is the right shape - it answers the operator's question on the affected host and stays quiet on sites that have never protected anything.
- The pool's `MAX_REQUESTS` recycle and the SM294 relay timeout (on the pending branch) both address the same class: a single bad request must not be able to take a worker, and therefore a site, down.

14.4 Evidence

- `docs/RELIABILITY.md:116-140` - the cadence rule and the rehearsal register.
- `CHANGELOG.md` - stable cuts at 0.8.0, 0.9.4, 0.9.10, 0.10.0.
- `docs/review/2026-07-18-eight-dimension/01-resolution.md:55-63` - the deferred items, both still open.

15 Dimension 6 - Security - lazysite eight-dimension review

- Audited tree: main at v0.10.8 (ec6fe0a)
- Date: 2026-08-14
- Regime: Commercial
- Prior verdict: REFUSE, cleared in-cut (2026-07-18)

15.1 Verdict

REFUSE. Three conditions, in descending order of seriousness:

1. A defect live in the released 0.10.8 edge build leaves content **stored as protected and still served** (F6.1 - the same defect as D1 F1.1, assessed here for its security consequence rather than its cause).
2. The **significant-change register has gone stale again** (F6.2), and it went stale over precisely the changes that fire its own triggers: content moved out of the document root, and a new external interface. This is the second consecutive review to raise the register.
3. The **threat model of record does not describe the current architecture** (F6.3). docs/SECURITY.md and docs/architecture/security.md contain no mention of the private store or the front door - the two structural changes that alter how protected content is stored and how requests are routed.

The pentest gate remains properly declared with a dated, auditable waiver (ADR 0007) and has not expired. The supply-chain controls are the right mechanisms, but the strict SBOM gate could not be run from the audited tag at all (F6.6).

This is a harsher verdict than the period's engineering deserves, and that should be said plainly: the 0.10.x line is the strongest sustained security work in the project's history. The refusal is about the *record* of that work keeping pace with it, plus one live bug.

15.2 Method

- Verified the 2026-07-18 refusal conditions as cleared or open.

- Read the significant-change register against the release history and against ADR 0007's own trigger list.
- Checked the STRIDE/ASVS documents against the architecture they claim to describe.
- Ran the SBOM strict gate and the secrets lint.
- Checked the SM283 disclosure's real-world closure state, not just its filing status.

15.3 Findings

15.3.1 F6.1 - Protected content is served after a crashed protect call (REFUSE)

Full mechanism at D1 F1.1. The security consequence, stated separately because it is what makes this a refusal rather than a bug report:

- the rule is stored and `acl-get` returns it, so every surface *reports* the content as protected;
- pages answer 302 to an anonymous request, which looks like working gating;
- **static files answer 200, byte-identical to the source;**
- the call never reaches the audit write, so the trail and the stored ACL disagree about whether anyone protected that content.

An operator inspecting this site through any interface lazysite offers is told the content is protected. It is not. That combination - a false negative in the audit trail and a false positive in the permission read - is worse than an outright failure, and it is the shape SM283 was.

Fix written and tested on `claude/sm296-acl-set-crash`; not in the audited tree.

15.3.2 F6.2 - The significant-change register is stale, over its own triggers (REFUSE)

ADR 0007 defers the first pentest engagement on the condition that the gate “fires ahead of schedule on significant change unless a recorded assessment finds the change contained”, with these triggers:

```
1 significant-change-triggers:
2   - new-external-interface
3   - new-authentication-method
4   - new-dependency-with-authentication-logic
5   - new-processing-of-restricted-data
```

The register's last entry is **2026-08-11 (SM279)**. Since that entry, 0.10.7 and 0.10.8 shipped the SM285-SM293 programme, which fires at least two triggers:

- **new-processing-of-restricted-data** - SM286 moves protected content out of the document root entirely, into `<docroot>-lazysite-private`. Where restricted content physically lives, and which identity may create and write that directory, changed.

- **new-external-interface** - SM293 step 5 ships `lazysite-front.pl`, a new CGI surface that a front end is pointed at and that makes every routing decision for the site.

Neither has a register entry. The waiver's integrity depends on this register being kept, because the register is the thing standing in for an engagement that has not happened. A deferral whose conditions are not being recorded is not a deferral.

This is the second consecutive review to raise it: F6.2 in the 2026-07-18 review was "significant-change register stale", closed in-cut. It went stale again within four weeks, which suggests the register needs to be part of the release checklist rather than a review artefact.

15.3.3 F6.3 - The threat model of record predates the architecture (REFUSE)

```
1 $ grep -c "private store\\|lazysite-private" docs/SECURITY.md
  docs/architecture/security.md
2 docs/SECURITY.md:0
3 docs/architecture/security.md:0
4 $ grep -c "FrontDoor\\|front door" docs/SECURITY.md
  docs/architecture/security.md
5 docs/SECURITY.md:0
6 docs/architecture/security.md:0
```

`docs/architecture/access-control-model.md` (SM290) does describe both, and is pinned to the code by `t/lint/36`. So the knowledge exists and is enforced - but it is not in the documents that `docs/POLICY.md` and the Declaration of Conformity point at as the security model of record, and a Commercial regime requires a current STRIDE/ASVS model for a user-facing service.

The gap is not cosmetic. The private store changes the answer to "where does restricted data live and who can read it", and the front door changes the answer to "what decides whether a request reaches the engine" - the two questions the threat model is for.

15.3.4 F6.4 - SM283's mitigation is shipped but not deployed where it was disclosed (WARN)

SM283 was a live disclosure: gated static files served anonymously by the front end, across a fleet, for weeks. The code remedy shipped (the Hestia proxy templates, the `X-Lazysite-Front` observable, `t/lint/33`, `t/integration/42`).

The filing itself is explicit that this is **not delivered by a package upgrade** - the template must be installed and each domain moved onto it. Probed from outside on 2026-08-14, `edge.explore.lazysite.io` - confirmed running 0.10.8 by fingerprint - returns **no X-Lazysite-Front header**, so the proxy template is not installed on that domain.

That is an operator step rather than a code defect, and the project built the observable precisely so it could be checked without credentials. But a mitigation for a live disclosure that is not deployed on the disclosing host is an open exposure, and it should be tracked as one until a fleet sweep says otherwise. Recorded as task #204; the fleet-wide check (#196) has also not been run.

15.3.5 F6.5 - What passes (PASS, noted)

- **The pentest waiver is intact and unexpired:** ADR 0007, signed and dated 2026-07-10, expiry the first third-party engagement or 2026-12-31, whichever first. That expiry is now four and a half months away and should be on the release manager's calendar.
- **SBOM strict gate:** the design is right - the SBOM cannot drift from the code, because a release fails if the code imports an undeclared module. But it **could not be executed on the audited tree**; see F6.6.
- **The in-app trust gate** (SM293 step 4, `t/lint/38`) converts a front-end configuration requirement into an enforced application control. This is the right direction and directly addresses the root cause behind SM248, SM268 H17 and SM283.
- **lazysite check --check-acl** lets a site prove its own gating from outside, with public controls of the same file type - so a refusal that happens because the front end cannot read the file is distinguishable from a refusal that is the ACL working.

15.3.6 F6.6 - The SBOM gate cannot be run from the tag it attests (WARN, new)

An earlier draft of this report listed the strict SBOM gate as passing. It was then run, and it does not:

```
1 $ perl tools/manifest-to-sbom.pl --strict
2 Cannot read /srv/projects/lazysite-audit/release-manifest.json: No
  such file or directory
3 sbom rc=2
```

Same root cause as D3 F3.1: `release-manifest.json` is a **gitignored build artefact**. It exists in a developer's working copy after a build, and does not exist in a clean checkout of `v0.10.8`.

For D3 this was a test-reproducibility problem. Here it is a compliance one, and sharper. The SBOM is a CRA Article 13 obligation, and the strict gate is the control the project relies on to keep it honest - `docs/POLICY.md` cites it as the reason the SBOM "cannot drift from the code". That claim is true during a release build and **unverifiable afterwards**: an auditor, a downstream consumer, or a future maintainer who checks out the tag cannot run the control that substantiates it.

This does not mean the SBOM is wrong. It means the gate's result is not reproducible from the artefact of record, which for a compliance control is the property that matters. Making `install.pl` and the SBOM tool generate the manifest when it is absent (it is derivable from the tree) fixes D3 F3.1 and this finding together.

15.4 Evidence

- `lib/Lazysite/Private.pm:223, :261.`
- `docs/SECURITY.md:72-105` - the register and its last entries.
- `docs/adr/0007-pentest-deferral.md:26-56` - triggers and waiver expiry.
- `curl -sI https://edge.explore.lazysite.io/ -no X-Lazysite-Front.`
- `tools/manifest-to-sbom.pl --strict` on a clean worktree - `rc=2`, cannot read `release-manifest.json`.

16 Dimension 7 - Documentation - lazysite eight-dimension review

- Audited tree: main at v0.10.8 (ec6fe0a)
- Date: 2026-08-14
- Regime: Commercial
- Prior verdict: WARN, cleared in-cut (2026-07-18)

16.1 Verdict

WARN, close to refusal on one item. The five-audience taxonomy is present and the reference documentation is in unusually good health - several documents are now *generated* or *lint-pinned to the code*, which is the strongest form this dimension can take. What fails is currency of the narrative documents: the feature timeline stops eight releases back (F7.1), and the policy document cites a review two generations old as the current one (F7.2).

F7.1 is the same item this dimension raised and cleared at the 0.8.0 gate. A documentation item that regresses within four weeks of being fixed is a process finding, not a writing finding, and is treated as such in the recommendations.

16.2 Method

- Checked the five-audience taxonomy for presence and for currency separately - presence has never been the problem here.
- Compared every narrative document's newest referenced version against the audited tree's version.
- Checked which documents are mechanically defended (generated, or pinned by a lint) versus maintained by hand.

16.3 Findings

16.3.1 F7.1 - The feature timeline stops at 0.9.14; the whole 0.10.x line is missing (WARN, regression)

docs/FEATURES.md Part XIII is the feature timeline, and its most recent entry is **0.9.14 (2026-07-24)**. Its closing summary states the document covers the project “to v0.9.14”. The audited tree is **0.10.8**.

Absent from it:

- **0.10.0 STABLE** (2026-07-27) - a stable promotion, the kind of release an operator reads this document to understand;
- the 0.10.1-0.10.6 line;
- **0.10.7 and 0.10.8** - which include SM283’s remedy and the SM285-SM293 programme, described in the changelog itself as “the largest structural change in the 0.10 line”.

The 2026-07-18 review recorded “FEATURES.md timeline swept to 0.7.28” as the D7 stable-gate item and cleared it. Four weeks later the same document is eight releases behind. The pattern says the sweep is being done as review remediation rather than as part of cutting a release, so it will keep regressing until it moves into the release procedure or is generated.

16.3.2 F7.2 - docs/POLICY.md cites the 2026-07-01 review as the current one (WARN)

docs/POLICY.md:34, in the CRA Article 13 obligations table - the project’s stated posture of record:

the 2026-07-01 eight-dimension review found WARN on several dimensions and its follow-up actions are in progress (see docs/review/2026-07-01-eight-dimension/)

Two further full reviews have happened since (2026-07-10 and 2026-07-18), and this is the fourth. The 2026-07-01 findings it describes as “in progress” were resolved in July. A compliance document that points a reader at a superseded assessment misrepresents the project’s posture in the conservative direction, which is the less harmful direction but is still wrong. docs/POLICY.md:64 carries the same stale pointer.

16.3.3 F7.3 - ADR 0001’s text no longer describes its own arrangement (WARN, carried from D1)

ADR 0001 speaks of “one recorded copy” of the module-free logic the render path carries. The audited tree has several, each now pinned by a lint that compares *answers* rather than text (t/lint/35, t/lint/37, and on the pending SM294 branch t/lint/42). The engineering is sound and

arguably stronger than the ADR describes; the ADR simply has not been rewritten to say so. Carried forward from the 2026-07-18 deferred list, where it was also not done.

16.3.4 F7.4 - The mechanically-defended documentation is the strong part (PASS, noted)

Worth recording because it is where this project has moved ahead of its own framework:

- docs/reference/capability-map.md is **generated**, and regeneration is required after any Capabilities.pm edit.
- docs/architecture/access-control-model.md is **pinned to the code** by t/lint/36, so the access model cannot drift from what is enforced.
- The **manager UI guide** is lint-enforced against the actual navigation (t/lint/32).
- t/lint/27 asserts that documentation references real paths, and t/lint/26 ties backlog status to the changelog, so a filing cannot claim to have shipped something a release does not mention.
- The **CHANGELOG** is exceptionally strong: each release explains the cause, not just the change, and several entries state plainly what an operator must do that a package upgrade will not do for them.

The distinction that matters for the recommendations: **every document in this list is defended by a mechanism, and every document in F7.1-F7.3 is maintained by hand.** The hand-maintained ones are the ones that rotted. That is the same finding this project has now made four times about hand-maintained *lists* in tests, applied to documents.

16.4 Evidence

- docs/FEATURES.md:1572 (newest timeline entry), :1808 (the “to v0.9.14” statement).
- docs/POLICY.md:34, :64.
- docs/review/2026-07-18-eight-dimension/00-overview.md - the D7 item cleared at the 0.8.0 gate.

17 Dimension 8 - Policy compliance - lazysite eight-dimension review

- Audited tree: main at v0.10.8 (ec6fe0a)
- Date: 2026-08-14
- Regime: Commercial (CRA Article 13 manufacturer duties)
- Prior verdict: WARN (2026-07-18)

17.1 Verdict

REFUSE, on two conditions.

The Declaration of Conformity is still drafted, unsigned, and stamped **0.8.0**, while the project has since shipped **three further stable releases** - 0.9.4, 0.9.10 and 0.10.0 (F8.1). Under the declared Commercial regime, a stable release is the artefact the declaration attaches to. Three have gone out without one.

Second, and with a nearer date: the posture of record addresses CRA **Article 13** in detail and **Article 14 not at all** (F8.6). There is no reference anywhere in the tree to the incident and actively-exploited-vulnerability reporting duties, no named accountable person for them, and no runbook. The project has already had one live exposure this period, so this is not a theoretical gap.

This is a paperwork refusal rather than an engineering one, and the distinction is worth stating: nothing here says the product is unsafe. It says the project is shipping stable releases faster than it is executing the compliance procedure it wrote for itself, and D8 is the dimension whose entire job is to notice that.

The supply-chain half of this dimension is in reasonable order: the licence position is clean, the support-period commitment is declared, and the strict SBOM gate is the right mechanism - though it cannot be run from the tag it attests (D6 F6.6).

17.2 Method

- Read docs/DECLARATION-OF-CONFORMITY.md for its version stamp and signature state.

- Compared that against the stable releases in CHANGELOG.md.
- Walked the CRA Article 13 obligations table in docs/POLICY.md item by item and checked each claim against the tree.
- Ran the strict SBOM gate.

17.3 Findings

17.3.1 F8.1 - Three stable releases have shipped against an unsigned 0.8.0 declaration (REFUSE)

docs/DECLARATION-OF-CONFORMITY.md:

Field	Value in the tree
Subtitle	“draft for the 0.8.0 stable release”
Version	“0.8.0 - placeholder, to be finalised at the 0.8.0 stable cut”
Unique identification	git tag v0.8.0, tarball lazysite-0.8.0.tar.gz
Place and date of issue	“To be completed at the 0.8.0 stable cut”
Signature	“(unsigned draft)”

Stable releases since: **0.9.4** (2026-07-19, changelog says “certified”), **0.9.10** (2026-07-21), **0.10.0** (2026-07-27).

The 2026-07-18 review recorded this as “DoC stamped 0.8.0 - signature is the operator action at the cut”, which was correct then: the cut had not happened. It has since happened three times over, and the declaration was neither advanced nor signed. Two of those three changelog entries describe themselves as certified, which the declaration does not support.

Remedy is the responsible person’s action, not an engineering one: finalise and sign a declaration for the current stable (0.10.0), and make advancing it part of the stable-cut procedure so it cannot be missed again. Note that the changelog’s own convention already distinguishes stable promotions clearly, so the trigger is unambiguous.

17.3.2 F8.2 - The obligations table is itself out of date (WARN, cross-referenced to D7 F7.2)

docs/POLICY.md : 34 describes the quality-and-documentation-floors obligation as partial, citing the **2026-07-01** review's WARNs as in progress. Those were resolved in July across two subsequent reviews. The posture-of-record document therefore under-reports the project's actual position on the one obligation where it has made most progress, while over-reporting nothing.

17.3.3 F8.3 - The pentest waiver expiry is approaching and is a policy commitment (WARN)

ADR 0007 defers the first third-party engagement with a hard expiry: **the first engagement by 2026-12-31, or the first GA marketing, whichever comes first**. That is four and a half months out. After expiry, an absent report is a refusal condition rather than a deferral.

Two things follow. First, procurement lead time for a CREST-CRT/OSCP/GIAC-GPEN third-party engagement is not short, so the engagement needs booking well before the date. Second, the waiver's ongoing validity depends on the significant-change register being kept, and it is not being kept (D6 F6.2) - so the waiver is currently weaker than the ADR intends.

17.3.4 F8.4 - Supply chain and licensing (PASS)

- **Strict SBOM gate**: the right mechanism form of the obligation. A release fails if the code imports a module not declared in `dist/config/sbom-deps.json`, so the SBOM cannot silently drift during a build. It could not be executed on the audited tree, because it needs a gitignored build artefact - so the claim holds at release time and is not reproducible from the tag afterwards (D6 F6.6).
- **Licence** MIT, with LICENSE, COPYRIGHT and THIRD-PARTY-NOTICES.md present and consistent.
- **SBOM** generated per release in CycloneDX and shipped in the tarball.
- **Support period** declared: five years from the first stable release (0.7.0), security fixes on the stable channel.
- **Coordinated vulnerability disclosure** in place via SECURITY.md.

17.3.5 F8.5 - Obligations still open, unchanged since July (noted, not new)

Obligation	Status
Annex VII technical file	pending

Obligation	Status
Signed releases (Sigstore/cosign)	pending
OpenChain 5230 + 18974 written policies	pending
CE marking	due 11 Dec 2027, noted, not applied

None is a refusal condition today. The Annex VII file and signed releases both become materially harder the longer the release history grows, which is the retrofit asymmetry this framework is built around - they are cheap now and expensive later.

17.4 Maintaining compliance

Everything above is remediation - fix these stale records. This section is the other half, and it is the half that decides whether the same review has to be written again in six weeks: the machinery for *keeping* compliance rather than restoring it.

17.4.1 F8.6 - CRA Article 14 (reporting) is absent from the posture of record (REFUSE)

docs/POLICY.md walks Article 13 manufacturer duties obligation by obligation. Article 14 - reporting actively exploited vulnerabilities and severe incidents - appears nowhere in the repository:

```
1 $ grep -in "article 14\|reporting\|ENISA\|CSIRT\|24 hour\|72 hour"
   docs/POLICY.md
2 (no matches)
```

The CRA phases its obligations. My understanding is that the Article 14 reporting duties apply from **11 September 2026** - four weeks from this review - while the bulk of the regulation applies from 11 December 2027, which is the date docs/POLICY.md already records for CE marking. **Both the date and the scope judgement need confirming by the legal review the DoC already says is required;** this finding is raised because the obligation with the *nearer* date is the one that is missing entirely, not because the interpretation is settled.

Three things follow, and none of them is a document:

- **A named accountable person.** The DoC names a function (“Responsible person, Open Digital CC”) and no individual for reporting. A 24-hour clock is met by someone reachable who knows the procedure, or it is not met.
- **A runbook.** There is no incident or reporting runbook in the tree at all - no docs / file matching *incident* or *runbook*. The path from “we have discovered a live exposure” to “notify whom, by

when, with what evidence” is currently unwritten.

- **A rehearsal.** The same argument D5 makes about restore rehearsals applies with more force here: an untested reporting procedure under a 24-hour clock is a plan, not a capability.

SM283 is the worked example and the reason this is not hypothetical. Gated static files were served anonymously across a fleet for weeks. Nothing suggests it was exploited - but had it been, the obligation would have been an early warning within 24 hours, and there was no mechanism to produce one.

17.4.2 F8.7 - The declared remediation SLAs have no evidence (WARN)

ADR 0007 declares remediation SLAs as part of the pentest posture: critical 72h, high 30d, medium 90d, low 180d, with retest required for critical and high.

Nothing records whether any of them has been met. There is no vulnerability register - no dated list of what was found, when, its severity, when it was fixed, and when it was retested. SM283’s remediation was in fact fast, and that cannot be demonstrated from the repository.

A declared SLA with no record is an assertion. One dated register turns the existing good behaviour into evidence, which is what an auditor, a customer questionnaire or an Annex VII file will each ask for. It is also the natural place for the Article 14 decisions to be recorded (“assessed, not actively exploited, no notification required” is itself a finding worth dating).

17.4.3 F8.8 - Dated obligations are scattered, and one is written relatively (WARN)

Every dated commitment lives in a different document, and no single view exists:

Obligation	Date	Recorded in
CRA Article 14 reporting applies	2026-09-11 (to confirm)	nowhere
Pentest waiver expiry	2026-12-31	docs/adr/0007-pentest-deferral.md
CE marking	2027-12-11	docs/POLICY.md
Support period ends	2031-07-10	docs/POLICY.md, relatively

The support period is written as “five years from the first stable release (0.7.0)”. 0.7.0 was cut 2026-07-10, so the commitment runs to **2031-07-10** - but a relative date decays the moment a reader has to

work out which release was first stable, and this project has already renumbered its stable line three times since. Write the absolute date beside the rule.

One dated obligations register, with an owner per row, is the artefact. It is also the natural thing to gate: a release-gate check that fails when a listed date is inside its lead time is the same mechanism this review recommends for the records, applied to the calendar.

17.4.4 F8.9 - Annex VII and signed releases get monotonically more expensive (WARN)

Both are listed pending (F8.5), and both have the property that deferring costs more than doing:

- **The Annex VII technical file is mostly assembly, not authorship.** Four eight-dimension reviews, the SBOMs, the ADRs, the DoC, the coverage and bench records and the test results already exist. Started now as an *index* over those artefacts, it is a short document that stays current as a by-product of work already happening. Started in 2027 it is an archaeology exercise across three release lines.
- **Signed releases cannot be applied retroactively.** Every release cut without Sigstore/cosign attestation is permanently unattestable. The cost of delay is strictly monotonic, and unlike most of this list it can never be paid down.

This is the retrofit asymmetry the framework is explicitly built around, and these are the two clearest instances of it in the project.

17.4.5 F8.10 - There is no per-project implementation document (WARN)

The framework expects each project to author its own implementation document from `IMPLEMENTATION-TEMPLATE.md` - regime assignment, posture, monitors, migration plan - and keep it inside the project. `find` returns nothing matching `IMPLEMENTATION` in the tree.

`docs/POLICY.md` covers part of the ground, which is probably why the absence has not been noticed across four reviews. But the template's monitors and migration-plan sections are exactly the two things this review found missing elsewhere (D5 F5.2's absent `MONITORS.md`, and the operator steps that package upgrades do not deliver). Authoring the implementation document would have surfaced both without a review.

17.5 Evidence

- `docs/DECLARATION-OF-CONFORMITY.md`: 3, :9-15, :26-27, :103-104.
- `grep -in "article 14|reporting|ENISA|CSIRT" docs/POLICY.md` - no matches.
- `CHANGELOG.md`: 2393 - 0.7.0 first stable, 2026-07-10.

- docs/adr/0007-pentest-deferral.md - the declared remediation SLAs.
- CHANGELOG.md - stable cuts at 0.9.4, 0.9.10, 0.10.0.
- docs/POLICY.md:29-45 - the Article 13 obligations table.
- docs/adr/0007-pentest-deferral.md:53-58 - the waiver expiry.
- tools/manifest-to-sbom.pl --strict on a clean worktree - rc=2, cannot read release-manifest.json.